

IdentityLearning Project Knowledge Base & Instructions

Project Overview

Project Name

IdentityLearning - Enterprise Authentication & User Management System

Purpose

A comprehensive ASP.NET Core Blazor application demonstrating enterprise-grade authentication, authorization, and user management capabilities using ASP.NET Core Identity, designed as both a learning platform and production-ready showcase.

Technology Stack

- **Framework:** .NET 9.0, Blazor Server-Side Rendering (SSR)
 - **Authentication:** ASP.NET Core Identity
 - **UI Framework:** MudBlazor 8.x
 - **Database:** SQL Server with Entity Framework Core
 - **Email Service:** Custom SOAP integration (AgileEmailService)
 - **Additional:** Extensions.MudBlazor.StaticInput, QR Code Generator
-

Architecture Overview

Solution Structure

```

IdentityLearning/
├── Components/
│   ├── Account/      # Identity-related components and pages
│   │   ├── Pages/    # Login, Register, Manage pages
│   │   ├── Shared/    # Shared Identity components
│   │   └── *.cs       # Identity services and helpers
│   ├── Layout/       # Application layouts
│   ├── Pages/        # Application pages
│   └── _Imports.razor # Global imports
├── Data/
│   ├── ApplicationDbContext.cs
│   ├── ApplicationUser.cs
│   └── Migrations/    # EF Core migrations
├── Services/
│   └── AgileEmailService.cs # Email service implementation
├── Properties/        # Launch settings and dependencies
├── wwwroot/           # Static files
├── Program.cs         # Application entry point
├── appsettings.json   # Configuration
└── IdentityLearning.csproj

```

Key Components

1. Identity System

- **ApplicationUser:** Extended IdentityUser with custom properties
 - Department (string)
 - IsActive (bool)
 - ColaboratorId (int?)

2. Email Service Architecture

- SOAP-based email service integration
- Support for HTML tables in emails
- Configurable SMTP settings
- Template-based email sending

3. Authentication Features

- Email/Password authentication
- Two-factor authentication (2FA) with QR codes
- Email confirmation
- Password reset functionality

- Account lockout protection
 - External authentication providers (ready for configuration)
-

Development Setup

Prerequisites

1. **.NET 9 SDK** or later
2. **SQL Server** (Express or Developer edition)
3. **Visual Studio 2022** (v17.14+) or VS Code
4. **Git** for version control

Initial Setup Steps

1. Clone and Configure

```
bash

# Clone the repository
git clone [repository-url]
cd IdentityLearning

# Restore packages
dotnet restore
```

2. Database Configuration

Update `appsettings.json` with your SQL Server connection:

```
json

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=YOUR_SERVER;Database=IdentityLearning;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}
```

3. User Secrets Setup

Configure sensitive data using User Secrets:

```
bash
```

```
# Initialize user secrets
dotnet user-secrets init

# Set email service credentials
dotnet user-secrets set "EmailSettings:ServiceKey" "your-service-key"
dotnet user-secrets set "EmailSettings:Password" "your-email-password"
```

4. Database Migration

```
bash

# Apply existing migrations
dotnet ef database update

# Create new migration (if needed)
dotnet ef migrations add YourMigrationName
```

5. Run the Application

```
bash

# Development mode
dotnet run --launch-profile https

# Or use Visual Studio F5 debugging
```

Feature Implementation Guide

Adding New Identity Features

1. Extending User Properties

```
csharp

// In Data/ApplicationUser.cs
public class ApplicationUser : IdentityUser
{
    public string? Department { get; set; }
    public bool IsActive { get; set; }
    public int? ColaboratorId { get; set; }
    // Add new properties here
    public DateTime? LastLoginDate { get; set; }
    public string? ProfilePicture { get; set; }
}
```

After adding properties:

```
bash

dotnet ef migrations add AddUserProperties
dotnet ef database update
```

2. Creating Custom Identity Pages

Place new pages in `Components/Account/Pages/`:

```
razor

@page "/Account/CustomPage"
@using Microsoft.AspNetCore.Identity
@inject UserManager<ApplicationUser> UserManager

<PageTitle>Custom Page</PageTitle>
<MudText Typo="Typo.h3">Custom Identity Page</MudText>

@code

{

    // Implementation

}
```

3. Adding Role Management

```
csharp

// In Program.cs
builder.Services.AddIdentityCore<ApplicationUser>(options => {
    // Configuration
})
.AddRoles<IdentityRole>() // Add this line
.AddEntityFrameworkStores<ApplicationDbContext>();
```

Email Service Integration

1. Email Configuration

The email service uses a SOAP endpoint. Configuration in `appsettings.json`:

```
json
```

```
{
  "EmailSettings": {
    "ServiceUrl": "https://axrm.agileincloud.com/agMailService/agEmailService.aspx",
    "ServiceKey": "", // Use User Secrets
    "SmtpServer": "smtp-pt.securemail.pro",
    "SmtpPort": 465,
    "UseSsl": true,
    "Username": "noreply@agileincloud.com",
    "Password": "", // Use User Secrets
    "FromEmail": "noreply@agileincloud.com",
    "FromName": "Your Company Name"
  }
}
```

2. Sending Emails with Tables

```
csharp

// Example of sending email with table
var emailBody = @"Order Confirmation

[TABLE:Order Details]
Product|Quantity|Price
Widget A|2|$10.00
Widget B|1|$15.00
[/TABLE]

Thank you for your order!";

await EmailService.SendEmailAsync(toEmail, subject, emailBody);
```

MudBlazor Components

1. Static Input Components

Use `MudStaticTextField` for SSR compatibility:

```
razor

<MudStaticTextField
  @bind-Value="Input.Email"
  For="@(() => Input.Email)"
  Label="Email"
  Placeholder="name@example.com" />
```

2. Form Handling

razor

```
<EditForm Model="Input" OnValidSubmit="HandleSubmit" FormName="unique-form-name">
  <DataAnnotationsValidator />
  <MudGrid>
    <!-- Form fields -->
  </MudGrid>
</EditForm>
```



Security Best Practices

1. Password Requirements

Configure in `Program.cs`:

csharp

```
builder.Services.AddIdentityCore<ApplicationUser>(options =>
{
    options.Password.RequiredLength = 8;
    options.Password.RequireDigit = true;
    options.Password.RequireUppercase = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireNonAlphanumeric = true;
});
```

2. Account Lockout

csharp

```
options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5);
options.Lockout.MaxFailedAccessAttempts = 5;
options.Lockout.AllowedForNewUsers = true;
```

3. Two-Factor Authentication

Already implemented with QR code generation. Users can enable via:

`/Account/Manage/TwoFactorAuthentication`

4. Security Headers

Add to `Program.cs`:

csharp

```
app.UseHsts();  
app.UseHttpsRedirection();
```

Testing Guidelines

1. Test Accounts

Create test accounts with different roles:

- Admin: admin@test.com
- User: user@test.com
- Manager: manager@test.com

2. Test Scenarios

- ☐ Registration with email confirmation
- ☐ Login with wrong password (lockout testing)
- ☐ Password reset flow
- ☐ 2FA setup and login
- ☐ Profile management
- ☐ Email delivery

3. Unit Testing Structure

```
csharp  
  
[TestClass]  
public class AuthenticationTests  
{  
    [TestMethod]  
    public async Task User_Can_Register_With_Valid_Data()  
    {  
        // Arrange  
        // Act  
        // Assert  
    }  
}
```

Configuration Management

Development Settings

```
appsettings.Development.json:
```



```
json
```

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Debug",
      "Microsoft.AspNetCore": "Warning",
      "IdentityLearning.Services": "Debug"
    }
  }
}
```

Production Settings

Use environment variables or Azure Key Vault:

```
bash
```

```
# Linux/Mac
```

```
export EmailSettings__ServiceKey="production-key"
```

```
# Windows
```

```
set EmailSettings__ServiceKey=production-key
```



Deployment Instructions

1. Publish Profile

```
xml
```

```
<Project>
  <PropertyGroup>
    <PublishProfile>Production</PublishProfile>
    <Configuration>Release</Configuration>
    <Platform>Any CPU</Platform>
    <TargetFramework>net9.0</TargetFramework>
    <RuntimeIdentifier>win-x64</RuntimeIdentifier>
  </PropertyGroup>
</Project>
```

2. Database Deployment

```
sql
```

```
-- Ensure database exists
```

```
CREATE DATABASE IdentityLearning;
```

```
-- Run migrations
```

```
dotnet ef database update --connection "Production Connection String"
```

3. IIS Deployment

1. Install .NET 9 Hosting Bundle
2. Create application pool (No Managed Code)
3. Set up website pointing to published folder
4. Configure environment variables

4. Azure Deployment

```
bash
```

```
# Using Azure CLI
```

```
az webapp up --name YourAppName --resource-group YourRG
```

Troubleshooting

Common Issues

1. Database Connection Errors

- Verify SQL Server is running
- Check connection string format
- Ensure database exists
- Verify user permissions

2. Email Service Failures

- Check SOAP endpoint availability
- Verify credentials in User Secrets
- Check firewall/network settings
- Review email service logs

3. Authentication Issues

- Clear browser cookies
- Check security stamp

- Verify email confirmation
- Review Identity logs

4. Migration Errors

```
bash

# Reset migrations if needed
dotnet ef database drop
dotnet ef migrations remove
dotnet ef migrations add Initial
dotnet ef database update
```

Monitoring & Logging

1. Application Insights (Azure)

```
csharp

builder.Services.AddApplicationInsightsTelemetry();
```

2. Serilog Configuration

```
csharp

builder.Host.UseSerilog((context, configuration) =>
    configuration.ReadFrom.Configuration(context.Configuration));
```

3. Custom Logging

```
csharp

_logger.LogInformation("User {Email} logged in at {Time}",
    user.Email, DateTime.UtcNow);
```

Maintenance Tasks

Daily

- Monitor error logs
- Check email delivery status
- Review failed login attempts

Weekly

- Database backup
- Security audit log review
- Performance metrics review

Monthly

- Update NuGet packages
 - Security patches
 - User access review
 - Certificate renewal check
-



Additional Resources

Documentation

- [ASP.NET Core Identity Docs](#)
- [MudBlazor Documentation](#)
- [Entity Framework Core](#)

Support Channels

- GitHub Issues: [Report bugs]
- Email: support@yourcompany.com
- Teams Channel: #identity-learning

Version History

- v1.0.0 - Initial release with basic Identity
 - v1.1.0 - Added 2FA support
 - v1.2.0 - Email service integration
 - v1.3.0 - MudBlazor UI upgrade
-



Important Notes

1. **Never commit secrets** to source control
2. **Always use HTTPS** in production
3. **Keep dependencies updated** for security
4. **Regular backups** are essential
5. **Monitor security advisories** for .NET Core
6. **Test thoroughly** before production deployment

7. **Document all customizations** for maintenance

8. **Follow GDPR/compliance** requirements

Quick Start Checklist

For New Developers

- ☐ Install prerequisites
- ☐ Clone repository
- ☐ Configure database
- ☐ Set up user secrets
- ☐ Run migrations
- ☐ Start application
- ☐ Create test account
- ☐ Review documentation

For Deployment

- ☐ Update connection strings
 - ☐ Configure email settings
 - ☐ Set environment to Production
 - ☐ Run database migrations
 - ☐ Configure SSL certificate
 - ☐ Set up monitoring
 - ☐ Verify email delivery
 - ☐ Test authentication flow
-

Contact Information

Project Lead: [Your Name]

Email: [your.email@company.com]

Teams: @YourHandle

Last Updated: August 2024

Version: 1.3.0

License: [Your License]